

IRWA Project Part-1: Text Processing and Exploratory Data Analysis

Authors: Carla Núñez u214970 / Julia Pérez u214026 / Jan Prats u213927

GitHub URL: https://github.com/Januto30/SearchEngine-IRWA_2025

TAG: 24/10/2025

"We have used ChatGPT to review our draft explanation of the pre-processing decisions and debug some functions"

PART 1: Data preparation

We have a document corpus, which is a JSON file containing an e-commerce fashion products dataset. This document follows a list-of-dictionaries structure. Each element in the list is a dictionary representing an individual product. Each product may include the following fields:

- Simple fields: `_id`, `actual_price`, `average_rating`, `brand`, `category`, `crawled_at`, `description`, `discount`, `out_of_stock`, `pid`, `seller`, `selling_price`, `sub_category`, `title`, and `url`
- Composite fields: `images` and `product_details`

The goal of the first part of the project is to preprocess the product text in order to prepare the data before applying techniques to analyse the text for classification, search, or recommendation purposes.

SETUP:

Since we have a list of dictionaries, we have loaded the data directly into a `DataFrame`. This automatically creates columns using the JSON document keys, and each row represents a product. By doing this, we avoid having to use for loops to access and process each field, as well as the inability to apply vectorized operations or perform tabular analysis easily. Additionally, following the HINTS provided by the instructors in the document *'IRWA_project_part_1.pdf'*, having a `DataFrame` will make it easier for us to refer to other documents such as *'validation_labels.csv'*.

PREPROCESSING FUNCTION:

We developed the function `build_terms(line)` based on the `build_terms(line)` function created in the classroom exercise *'01_Model_Tf_IDF_Index_GRUP'*. What our function does to preprocess the document, specifically the product text (*title*, *description*), is:

- Transform text to lowercase.
- Remove punctuation marks and non-alphanumeric elements (while keeping spaces)
- Collapse multiple spaces into a single space.
- Tokenize the text to obtain a list of terms.
- Remove stopwords.
- Perform stemming.

After applying the `build_terms` function, we obtain two new columns in our DataFrame (*title_clean* and *description_clean*), which contain lists of normalized terms ready for analysis. On the other hand, the field `product_details` contains a list of dictionaries with attributes for each product, for example:

```
"product_details": [  
  {  
    "Style Code": "1005COMBO2"  
  },  
  {  
    "Closure": "Elastic"  
  },  
  {  
    "Pockets": "Side Pockets"  
  },  
  {  
    "Fabric": "Cotton Blend"  
  },  
  {  
    "Pattern": "Solid"  
  },  
  {  
    "Color": "Multicolor"  
  }  
]
```

To integrate them into the textual analysis, we defined the function `combine_product_details()`, which:

- Verifies that the value is a list of dictionaries.
- Extracts the values of each attribute and converts them into text.
- Combines all values into a single string, separating the terms with spaces.

The new column `product_details_text` in our DataFrame contains a textual representation of all product details, allowing us to treat these structured data as plain text comparable to the *title* and *description* fields.

This preprocessing ensures that the product texts are normalized, noise-free, and ready to be compared with search queries.

PREPROCESSING FOR EXAMPLE:

Once the preprocessing functions were defined, we applied them to all product entries in the dataset. The goal of this step is to systematically clean and tokenize the textual content of each product (title and description), storing the processed tokens for subsequent analysis.

We created an empty list, *preprocessed_products*, which stores all processed items. Then, for each product in the JSON dataset:

- The title and description fields are extracted.
- Both fields are passed through the preprocessing pipeline using the *build_terms()* function.
- A new dictionary is created, *prod_processed*, containing all original product fields plus two new ones:
 - *title_tokens*: the list of processed terms from the product title.
 - *description_tokens*: the list of processed terms from the product description.
- The processed dictionary is appended to the list *preprocessed_products*.

Finally, the code prints an example:

=== Original Product ===

Title : Solid Women Multicolor Track Pants

Description: Yorker trackpants made from 100% rich combed cotton giving it a rich look. Designed for Comfort, Skin friendly fabric, itch-free waistband & great for all year round use Proudly made in India

=== After Preprocessing ===

Title Tokens : ['solid', 'women', 'multicolor', 'track', 'pant']

Description Tokens: ['yorker', 'trackpant', 'made', '100', 'rich', 'comb', 'cotton', 'give', 'rich', 'look', 'design', 'comfort', 'skin', 'friendli', 'fabric', 'itch', 'free', 'waistband', 'great', 'year', 'round', 'use', 'proudli', 'made', 'india']

Showing the first product before and after preprocessing, displaying its raw text and the resulting tokenized form.

EXPLORATORY DATA ANALYSIS:

After completing the text preprocessing stage, we performed an exploratory data analysis to better understand the characteristics of the dataset. This phase helps visualize the content, distribution, and structure of product information before moving on to text modelling or retrieval tasks.

1. Word Cloud Visualization

To obtain an overview of the most frequent terms appearing in product titles and descriptions, we concatenated all cleaned tokens from both fields into a single string and generated a word cloud.

```
text_combined = " ".join([" ".join(x) for x in df_products['title_clean']] +
                          [" ".join(x) for x in df_products['description_clean']])
wordcloud = WordCloud(width=800, height=400,
                      background_color='white').generate(text_combined)
```

2. Price Distribution

Next, we explored the numerical features related to product pricing. Both `selling_price` and `actual_price` fields were converted to numeric values after removing commas and coercing non-numeric entries.

```
df_products['selling_price'] = pd.to_numeric(df_products['selling_price'].str.replace(',', ''),
errors='coerce')
df_products['actual_price'] = pd.to_numeric(df_products['actual_price'].str.replace(',', ''),
errors='coerce')
```

A histogram was then plotted to display the distribution of selling prices, showing how prices vary across products and allowing the identification of possible outliers or skewed price ranges.

3. Average Rating Distribution

Similarly, the `average_rating` field was converted to a numeric format and visualized using a histogram. This analysis provides insights into the general satisfaction level of products and the rating patterns across the dataset.

4. Top Brands and Categories

To identify the most represented brands and product types in the dataset, we computed frequency counts of both the brand and category fields and visualized the Top 10 of each using bar plots.

```
top_brands = df_products['brand'].value_counts().head(10)
top_categories = df_products['category'].value_counts().head(10)
```

This step reveals the most common fashion brands and categories, helping to understand market representation and dataset bias.

5. Stock Availability

We also examined the `out_of_stock` field through a count plot, displaying how many products are currently available versus out of stock.

This metric provides a simple but useful view of inventory distribution across the dataset.

6. Exporting the Pre-processed Dataset

Finally, a subset of relevant pre-processed columns was selected and saved into a new CSV file named `preprocessed_fashion_products.csv`, which includes cleaned text fields, key metadata, and numerical attributes.

```
preprocessed_cols = ['pid', 'title_clean', 'description_clean', 'brand', 'category', 'sub_category',
'product_details_text', 'seller', 'out_of_stock', 'selling_price', 'discount', 'actual_price',
'average_rating', 'url']
df_products[preprocessed_cols].to_csv("preprocessed_fashion_products.csv", index=False)
```

This file serves as the finalized dataset for further stages of the IRWA project, such as feature extraction, vectorization, and similarity-based product search.

PART 2: Exploratory Data Analysis

1. Word Cloud

We used a word cloud to visualize the most common terms in product titles and descriptions. This type of visualization is especially helpful for gaining a quick, intuitive understanding of the dataset's dominant vocabulary, highlighting the words that appear most frequently after preprocessing.

From the visualization, the most important words include *"men," "women," "shirt," "round," "neck," "fit," "slim,"* and *"solid."* This indicates that the dataset is heavily focused on apparel products, especially tops and shirts. The repetition of material-related terms like *"cotton"* and *"blend"* suggests that fabric quality and comfort are major selling points. The interesting thing is that these high-frequency terms align closely with the validation queries (e.g., *"women full sleeve sweatshirt cotton," "men slim jeans blue"*), which confirms that our preprocessing retains the essential fashion-related vocabulary necessary for accurate retrieval performance.

2. Histogram of Selling Prices

A histogram was chosen to represent the distribution of selling prices, as it can reveal how product prices are spread across different ranges and highlights skewness in the data. Understanding this distribution can help us detect pricing patterns, outliers, and potential data biases that might influence ranking or retrieval strategies.

The resulting plot shows a clear right-skewed distribution, where most products are priced below 1.000 euros, and only a few extend to higher price ranges. This indicates that the dataset mainly consists of affordable and mid-range fashion items, with a limited presence of luxury items. The long tail of higher-priced items suggests the inclusion of some premium or branded products. This insight can be relevant for retrieval systems for instance, price could be used as a secondary ranking factor to align with user intent (filtering by premium or affordable products)

3. Histogram of Average Ratings

We used another histogram to visualize the distribution of average product ratings, since this type of plot helps us quickly see central tendencies and spread in numeric variables.

The plot shows that most ratings fall between 3.5 and 4 points out of 5, indicating that users generally perceive these products as satisfactory or above average. Few products have extremely low (below 2) or perfect (5) ratings, suggesting realistic and balanced feedback. We interpret that the slight skew toward positive ratings is common in online shops, where satisfied customers are more likely to leave reviews.

4. Bar Chart of Top Brands

We decided to use a bar chart to analyse the frequency of products by brand, as this visualization clearly conveys category-level dominance and allows easy comparison among the top brands and allows us to comprehend possible biases in the data.

The results show that only a few brands (*"ECKO Uni," "Free Authority," "ARBO," "REEB,"* ..) account for a large portion of the dataset, while many other brands appear infrequently. This reflects a long-tail brand distribution, where a few big brands dominate listings. This pattern is important for information retrieval because brand names often appear repeatedly and can overshadow other relevant text signals.

5. Bar Chart of Top Categories

For analysing the top categories, we also selected the bar chart as the best option for visualization as it makes it easy to detect category imbalance and product concentration.

The plot reveals that the “Clothing and Accessories” category dominates the dataset, followed by much smaller portions of “Footwear” and a few other categories like “Bags, Wallets & Belts” or “Toys.” This strong imbalance confirms that the dataset primarily represents fashion apparel, which aligns with the overall project context. However, it also suggests limited diversity in other product types. From a modelling perspective, this insight supports the idea of assigning greater weight to sub-categories and descriptive fields during indexing, since they contain more discriminative terms (“jeans,” “track pants,” “t-shirt”, etc) that help differentiate between similar clothing items.

6. Bar Chart of Out-of-Stock Products

To analyse product availability, we used a bar chart again to visualize the proportion of products that are in-stock versus out-of-stock.

The plot shows that most products are available, while only a smaller share is currently out-of-stock. This suggests that the dataset reflects an active and maintained catalog, representative of a real e-commerce environment. Although the out-of-stock flag doesn’t influence textual similarity directly, we think it can become important during post-retrieval filtering, ensuring that only available products are shown to users in a live search system.